感悟报告

这次超算队的考核对我来说是一次很大的成长。

基础部分我觉得前几关还是比较简单的，但是确实也蛮好玩，虽然自己其实并不是每关都很顺利地做出了想要的结果，但是总的来说感觉很有意思。印象最深的是第 1 关那个"必须是普通文件，不能是符号链接"的报错：我本来还纳闷 input.bin 明明就在那，问了 AI 才知道是软链接在作怪，删掉软链接、用 cp 复制出普通文件才给过——感觉出题人连防走捷径都考虑到了。不过 git 那部分就得好好反省一下了，由于我的轻敌，明知道基础部分除了 Linux 其实还有 git 的内容，但我觉得自己之前已经用过 git 了，就没做完基础部分（甚至 git 的内容根本就没做）就直接去进阶了 🙏 。结果第一次推送代码就翻车，先栽在分支上，又栽在认证上，黑框卡死连粘贴都不行，折腾了好久才推上去。这件事让我真切体会到，"以为会了"和"真的会了"之间隔得很远，之前的自己太轻敌了。

进入 CUDA 阶段后，一维的向量加法到二维的矩阵，再到共享内存和卷积的 Halo 边界，我也切实体会到了 CPU 和 GPU 思维的不同。 AI 说CPU 是"一个人慢慢算"，GPU 是"成千上万个人同时算"，这个我之前其实有所了解，但是直到现在写了一些关于它们的代码，才能真正体会到这些差距。

具体到题目上，这个东西是一步步练出来的：第 10 题矩阵加法练的是二维线程映射和边界保护，要算好全局行列坐标，`row < M && col < N` 的越界检查一个都不能少，grid 两个方向都得向上取整把矩阵盖住；第 11 题共享内存矩阵乘，我先写了直接读全局内存的朴素版本，后来再分块优化——朴素版本里 C 的每个元素要循环 K 次，每次都从全局内存各读一个数，全局访存次数是 $O(N^3)$ 量级的，分块之后 A、B 的每个 Tile 只需要各读一次，就能被整个 Block 里的线程反复复用；到了卷积又多出一个 Halo 边界的概念，共享内存 Tile 的尺寸要开成 `TILE_SIZE + 2*HALO`，把中心像素周围一圈也装进来，边界像素才算得对。

说到 CUDA 我感觉其实自己挺菜的。前面比较简单的几个，比如 Hello CUDA、向量加法、矩阵加法是我自己写的，后面的题目代码不少是 AI 生成的。不过也不是白靠 AI——在问性能优化的时候我学到了延迟和吞吐量这两个指标，明白了 GPU 也不是"无脑快"，它也受限于数据拷贝时间和线程启动开销，真正的优化要盯着访存模式看。具体内容我写在学习报告里了。

有几个体会值得单独说说：向量加法这种题目每次访存都很独立、没有复用，属于吞吐量型任务，GPU 很容易跑满；而矩阵乘、卷积这种有大量数据复用的，就更要关心每次访问要等多久（延迟），所以共享内存才有用武之地。还有 Bank Conflict——共享内存被划成了很多个 bank，如果同一个 warp 里的多个线程同时访问同一个 bank，访问就会被串行化，本来是片上缓存的速度优势就没了。这些细节让我意识到，性能优化不是玄学，是可以拆开一项项分析的。

说起来还有令人心碎的环境配置和 Git 推送。那个密码输入框卡死的问题折磨了我很久，一开始用的 git bash ，但是一直不能解决问题，后面换了powershell才解决 🙏 。在 WSL 里找 D 盘路径失败时，我也一度怀疑人生，再加上考核平台一直有点问题，CUDA 我没能在平台上真正跑一遍，也没有对应的截图，最后只

能让 AI 生成一些数据跑一下看看。后来我想通了：考核看的是我的代码和解决问题的思路，不是看我能不能把一个完美无缺的本地环境配好。所以后面我转去 Google Colab 。

现在回过头看，虽然过程很曲折，但是也学到了不少之前没真正接触过的知识。虽然自己已经大二了，但其实还是有很多不会的知识，自己就是个半瓶子水晃荡。而且由于自己之前的懒惰等等原因，似乎也没有很拿得出手的技术，期待能在超算队里鞭策自己，学到更多。